

setup_path_install

Design: `install_helix_path.sh` — PATH-install for HelixCode power sub- systems

Track (T1/feature/helixllm-full-extension). \$11.4.70 design pass.

Investigation findings (real build targets, cited file:line)

All paths below are relative to the HelixCode meta-repo root (/home/milos/Factory/projects/tools_and_research/helix_code).

Component	Build target	Real output binary	Evidence
HelixCode (inner app)	helix_code/ Makefile → make build	helix_code/ bin/helixcode	helix_code/Makefile:4 (BINARY_NAME=helixcode), helix_code/ Makefile:98-101 (\$ (GO_BUILD) ... -o bin/ \$(BINARY_NAME) ./\$(CMD_DIR)/server)
HelixCode CLI client	no dedicated Makefile target (see open question below)	helix_code/ bin/cli (best- effort direct go build)	helix_code/cmd/cli/ contains real source (main_commands.go, root.go, ...) but helix_code/Makefile has no bin/cli/cli: recipe (grepped, zero hits)
HelixAgent	submodules/ helix_agent/ Makefile → make build	submodules/ helix_agent/ bin/helixagent	submodules/ helix_agent/ Makefile:23-25 (build: → go build -mod=mod -ldflags="-w -s" -o bin/helixagent ./cmd/ helixagent)

Component	Build target	Real output binary	Evidence
HelixLLM	submodules/ helix_llm/ Makefile → make build	submodules/ helix_llm/bin/ helixllm	submodules/helix_llm/ Makefile:4 (BINARY := helixllm), submodules/ helix_llm/ Makefile:11-12 (build: → go build \$(GOFLAGS) -o bin/\$(BINARY) ./ cmd/helixllm)
LLMsVerifier	submodules/ llms_verifier/ Makefile → make build	submodules/ llms_verifier/ bin/llm- verifier	submodules/ llms_verifier/ Makefile:35-36 (build: → go build -o bin/llm- verifier ./cmd)

Open question (honest gap, §11.4.6 — not guessed, not invented)

helix_code/cmd/cli/ is a real, compilable command package (per CLAUDE.md §3.2.1’s own claim “CLI client entry → bin/cli”), but the **inner Makefile does not wire a target that produces bin/cli** — make build only builds the server binary (bin/helixcode). This is a genuine gap between the documented architecture (§3.2.1) and the actual Makefile. The install script therefore treats bin/cli as **best-effort**: it attempts a direct go build -o bin/cli ./cmd/cli (not make-driven) and reports it distinctly so the INSTALLED/MISSING table never silently claims a Makefile target that does not exist. If a maintainer later adds a real cli: Make target, the script’s Makefile-target step should be preferred over the direct-build fallback (left as a documented follow-up, not invented here).

Design

install_helix_path.sh (POSIX-ish bash, set -euo pipefail):

1. **Resolve repo root from script location** (§11.4.28/§11.4.177 decoupling) —
SCRIPT_DIR="\$(cd "\$(dirname "\${BASH_SOURCE[0]}")" && pwd)".
The script is designed to be *placed* at the HelixCode repo root (helix_code/
install_helix_path.sh), so REPO_ROOT="\$SCRIPT_DIR" by default,
overridable via HELIX_REPO_ROOT env var — no hardcoded absolute host
path.
2. **Component table** (bash associative-array-free — POSIX arrays for
portability): for each of the 5 build units above, records name|build_dir|
make_target|expected_bin_relpath|final_bin_name.

3. **Build phase** — for each component: if the expected binary already exists AND `--skip-build` not passed, skip; otherwise `cd "$build_dir" && make "$make_target"` (or the direct go build fallback for cli, clearly logged as “non-Makefile-driven”). Build failures are captured (stdout+stderr to a per-component log under `$REPO_ROOT/qa-results/path_install/<ts>/<component>.log`, §11.4.5/§11.4.69 evidence) and reported, but do not abort the whole run (continue with the other components) — final exit code reflects whether ANY component failed.
4. **Install phase (no sudo, user-writable PATH)** — target dir `{HELIX_BIN_DIR: ~$HOME/.local/bin}` (`mkdir -p`; no root, §11.4.161 spirit). For each successfully-built binary, create/refresh a symlink `$HELIX_BIN_DIR/<final_bin_name> -> <absolute path of built binary>` (`ln -sf`, so a rebuild is picked up automatically — no stale copy).
5. **Idempotent PATH export** — appends `export PATH="$HOME/.local/bin:$PATH"` to the user’s shell rc (`~/.bashrc` and `~/.zshrc` if present, else `~/.profile`), guarded by a sentinel comment (`# >>> HelixCode PATH (managed) >>> /# <<< HelixCode PATH (managed) <<<`) so re-running the script never double-appends — it rewrites the block between the sentinels if already present (classic idempotent-block pattern).
6. **Anti-bluff verification (§11.4.80 lesson: install exit 0 ≠working binary)** — for every symlinked binary: (a) command `-v <name>` (in a subshell with `PATH="$HELIX_BIN_DIR:$PATH"`, since the *current* shell may not have resourced its rc yet); (b) execute a lightweight health probe — try `<bin> --version`, falling back to `<bin> version`, falling back to `<bin> --help` (first one that exits 0 wins) — and capture the first output line as proof-of-life. Never assume; if all three probes fail the component is reported **BUILT-BUT-NOT-VERIFIED**, distinct from **MISSING** (honest 3-state result, §11.4.6).
7. **Report** — final per-component table to stdout: **INSTALLED** (built + symlinked + verified), **BUILT-BUT-NOT-VERIFIED**, **BUILD-FAILED**, or **MISSING** (source dir not found — e.g. submodule not checked out). Exit code = 0 only if every mandatory component (all except the best-effort cli) is **INSTALLED**.
8. **No secrets, no CI wiring** (§11.4.156) — pure local script, never invoked by any CI hook.

Usage (once placed at HelixCode root)

```
./install_helix_path.sh
    # build missing + install + verify
./install_helix_path.sh --skip-build    # install/verify only,
    using existing bin/*
HELIX_BIN_DIR=/custom/bin ./install_helix_path.sh
```

Files produced by this design pass (scratch only, per task constraint)

- `design_setup_path_install.md` (this file)
- `install_helix_path.sh` (complete, `bash -n`-clean, ready to be reviewed + placed at the HelixCode repo root by the conductor/operator — NOT placed automatically by this design subagent, per the “write only to scratch” constraint)

Sources verified

Sources verified 2026-07-08: this is an internal HelixCode design document describing a repo-internal PATH-install script for HelixCode’s own build outputs. Its factual claims (Makefile targets, binary output paths, build-dir layout) are verified directly against this repository’s own Makefiles — see the “Investigation findings” table above, which cites the exact `file:line` for every claim (`helix_code/Makefile`, `submodules/helix_agent/Makefile`, `submodules/helix_llm/Makefile`, `submodules/llms_verifier/Makefile`). No third-party service, library, SDK, or public API is documented in this file, so there is no external online source to cross-reference per §11.4.99; the repository itself (via CodeGraph/grep-verified `file:line` citations) is the authoritative source for every instruction here.